PROJECT DOCUMENTATION

Data Contract Validator and Reliable Pipeline

Technologies: Python • CSV • SQLite • pytest • Visual Studio Code

Contents

• Introduction and problem statement

• 1. Project objectives and workflow

• 2. Technologies used

• 3. Project folder structure

• 4. Data contract and validation rules

• 5. Database tables

• 6. Code explanation and execution steps

• 7. Testing and sample results

• 8. Advantages, limitations, and future scope

• 9. Conclusion

Introduction and Problem Statement

The project is called **Data Contract Validator and Reliable Pipeline**. It is a small data-processing program written in Python. It reads customer and order information from CSV files, checks whether the information follows agreed rules, and saves accepted records in an SQLite database.

In a real business, order files may contain mistakes such as a negative amount, an unknown customer ID, an unsupported status, or an invalid date. If incorrect information is stored without checking, it can lead to inaccurate reports and business decisions.

This project addresses that problem by checking the input before saving it. Valid records are stored in database tables. Invalid records are saved in a separate **rejects** table with the reason they failed. A summary is printed in the terminal.

> **Important:** “Completed with rejects” means the program finished processing and found invalid rows. It does not mean the program crashed.

1. Project Objectives and Workflow

Objectives

• Read customer and order data from CSV files.

• Check that required column names are present before processing rows.

• Validate important values such as IDs, regions, order amounts, statuses, and timestamps.

• Save valid customer and order records in SQLite.

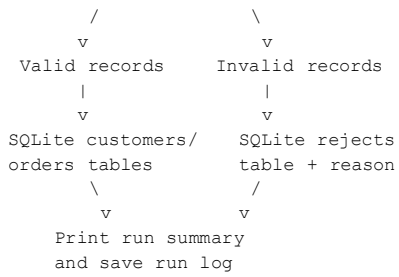
• Keep rejected rows and readable rejection reasons for later review.

• Handle repeated order IDs without creating duplicate order records, and avoid replacing a newer order with an older update.

• Record a run summary and use automated tests to check key behaviour.

Workflow

```
customers.csv + orders.csv
          |
          v
Read both CSV files
          |
          v
Check required column names
          |
          v
Validate each row and its values
```



The program processes customers first because each order must refer to a customer that exists. It then validates orders, writes the accepted data, records rejected rows, and prints totals and processing time.

2. Technologies Used

Technology	How it is used in this project
Python	Runs the program, reads data, checks rules, handles errors, and connects to SQLite.
CSV	Stores input data in simple rows and columns. The input files are customers.csv and orders.csv.
SQLite	Stores the accepted data, rejected rows, and run history in a local database file. It does not need a separate database server.
pytest	Runs automated tests that check important cases, including invalid rows, missing columns, and duplicate handling.
Visual Studio Code (VS Code)	Used to open the project, edit Python files, run terminal commands, and optionally inspect SQLite tables with an extension.

3. Project Folder Structure

File or folder	Purpose
src/pipeline.py	Main program: reads files, validates data, stores records, writes rejects and run logs, and prints the summary.
src/generate_data.py	Creates larger sample CSV files for optional performance testing. It can include deliberately invalid examples.
data/input/	Contains the small sample customer and order CSV files.
data/generated/	Destination for generated sample files when the data generator is run.
data/output/	Destination for SQLite database files produced by the pipeline.
tests/test_pipeline.py	Contains automated tests for the pipeline.
docs/data_dictionary.md	Explains the CSV columns and database tables.
requirements.txt	Lists pytest, the testing package needed by the project.

4. Data Contract and Validation Rules

A **data contract** is an agreement about what data should look like. In this project, it defines the required columns and acceptable values before records can be saved.

Required CSV columns

File	Required columns
customers.csv	customer_id, customer_name, region
orders.csv	order_id, customer_id, order_timestamp, amount, status, updated_at

Validation rules

Field / rule	What the program checks
Column names	All required column names must exist. Extra columns are allowed. If a required column is missing, the pipeline stops before processing rows.
customer_id	Must not be blank. Repeated customer IDs in the same input file are rejected.
customer_name	Must not be blank.
region	Must be one of north, south, east, west, or central. Letter case is normalised.
order_id	Must not be blank. Repeated order IDs in the same input file are rejected.
customer_id in orders	Must match a customer ID known to the database after customer processing.
amount	Must be numeric and zero or greater. Values are rounded to two decimal places before storage.
status	Must be pending, paid, shipped, cancelled, or refunded.
timestamps	Both order_timestamp and updated_at must be valid ISO-format dates/times. updated_at cannot be earlier than order_timestamp.
Order updates	When the same order already exists, an older update is ignored; a newer or equal update can replace the stored version.

If a row fails one or more checks, the program stores its file name, row number, record ID, and reason in the rejects table. This makes it easier to investigate bad data instead of silently discarding it.

5. Database Tables and Example SQL Queries

The program creates four tables in an SQLite database file, normally at data/output/pipeline.db.

Table	What it stores
customers	Accepted customer IDs, names, and regions. customer_id is the primary key.
orders	Accepted order details. order_id is the primary key; customer_id refers to customers.
rejects	A record of rejected rows, including the source file, row number, record ID, and reason.
run_logs	A summary of each run, including time, status, number of rows read/accepted/rejected, and duration.

6. Code Explanation and Execution Steps

Main code: src/pipeline.py

• **read_csv()** opens a CSV file and returns its column headings and rows.

• **check_columns()** checks whether all required headings exist and reports missing or extra headings.

• **validate_customer()** checks each customer ID, name, and region.

• **validate_order()** checks the order ID, known customer, amount, status, and timestamps.

• **create_database()** creates the four tables if they do not already exist.

• **save_reject()** writes a rejected row and its reason to the rejects table using a parameterised SQL statement.

• **run_pipeline()** coordinates the workflow, saves accepted records, logs the run, and prints the final summary.

• **main()** accepts optional file and database paths from command-line arguments.

Setup and run in VS Code on Windows

```
# 1. Open the project folder in VS Code, then open Terminal

# 2. Create a virtual environment (use an installed Python version)
py -3.12 -m venv .venv

# 3. Activate it in PowerShell

   .\.venv\Scripts\Activate.ps1

# 4. Install the project dependency
python -m pip install -r requirements.txt

# 5. Run the sample pipeline
python -m src.pipeline

# 6. Run automated tests
python -m pytest -q
```

If Python 3.12 is not installed, replace 3.12 with a version available on your computer.

Optional larger-data demonstration

```
python -m src.generate_data --orders 100000
python -m src.pipeline --customers data/generated/customers.csv --orders data/generated/orders.csv --db data/output/large_te
```

The second command writes to a separate database called large_test.db so the normal sample database is not used. Processing time depends on the computer; report the actual time shown by your own run.

7. Testing and Sample Results

Automated tests

I ran the supplied test suite while preparing this documentation. The result was **3 passed**. The tests cover these behaviours:

• A valid order is saved while an order with a negative amount is rejected.

• If required CSV columns are missing, the pipeline returns a schema error and stops.

• Running the same valid order input twice does not create a duplicate order row.

> Test result from this documentation run: **3 passed in 0.10 seconds**. Execution time can vary by computer and environment.

Sample input and output

The supplied sample has 4 customer rows and 5 order rows. The customer file has all required columns. In the order file, two records are valid; three contain issues: a negative amount, an unknown customer ID, and an unsupported status.

Measure	Observed result
Customer rows read	4
Customer rows accepted	4
Order rows read	5
Orders accepted/updated	2
Rows rejected	3
Customers in database after run	4
Orders in database after run	2
Rejected rows saved	3
Final status	COMPLETED WITH REJECTS

The sample pipeline run in this environment reported approximately 0.014 seconds. This is only the timing from this documentation run; timing can differ on another machine.

How to inspect the results

• Run python -m src.pipeline from the project root.

• Open data/output/pipeline.db with a suitable SQLite extension in VS Code.

• Inspect the customers, orders, rejects, and run_logs tables.

• Use SELECT * FROM rejects; to read the reasons for the three rejected sample orders.

8. Advantages, Limitations, and Future Scope

Advantages

• Easy to understand and run for a beginner.

• Finds common data problems before saving records.

• Does not silently lose rejected rows; it records why they failed.

• Uses SQLite, so no separate database server is needed.

• Uses automated tests to check key behaviours.

• Can create larger datasets for a basic scale demonstration.

Limitations

• The program reads CSV data and processes it on one computer; it is not a cloud or real-time streaming system.

• The validation rules are fixed in the Python code, so changing them requires code changes.

Future scope

• Add more tests for missing values, malformed timestamps, duplicate customers, and boundary amounts.

• Move validation rules to a configuration file so they can be changed without editing the main code.

9. Conclusion

This project demonstrates a simple but useful data engineering idea: validate data before trusting it. Python reads the customer and order CSV files, checks their structure and values, saves accepted records in SQLite, and records rejected rows with clear reasons. pytest helps verify that important behaviours work as expected.